8-2 · DAG 이해 및 유즈케이스 연구 · 실습 1.5H

첫 DAG 작성 — PythonOperator

DAG 세 개로 순서·병렬·재시도·실패를 직접 겪는다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① hello_dag 를 dags/ 에 저장하고 reserialize 로 바로 읽힌다

② af tasks test 로 태스크 하나를 단위 실행한다

③ etl_practice 를 unpause → trigger 로 수동 실행한다

④ 중간 태스크를 실패시키고 tasks clear 로 그 자리부터 다시 돌린다

사전 준비 — 「실습점검 · 환경재설정」 1 · 2단계(Airflow healthy · af 함수)

실습 흐름

아홉 단계 — ~/airflow-lab/dags 에 파일 세 개

dags/ 는 컨테이너 안 /opt/airflow/dags 에 마운트 — 저장하면 Airflow 가 읽어 갑니다

1·2. hello_dag	3. tasks test	4·5. etl_practice	6·7. 실패·clear	8·9. fan_practice
파일 저장 파싱 오류 내기	태스크 하나 단위 실행	순서 DAG unpause·trigger	raise 넣기 실패 지점부터	병렬·합류 순환 오류

핵심 명령

af 함수 하나로 — 스케줄러 컨테이너의 airflow

하려는 일	명령
새 파일 바로 읽기	af dags reserialize
파싱 오류 보기	af dags list-import-errors
단위 실행	af tasks test <dag> <task> <날짜>
활성화 · 수동 실행	af dags unpause · af dags trigger
실행 목록	af dags list-runs <dag>
태스크별 상태	af tasks states-for-dag-run <dag> "$RUN"
실패한 곳부터	af tasks clear <dag> -t <task> -d -y

af = docker compose exec airflow-scheduler airflow — 실습점검 2단계에서 이 터미널에 만든 함수

1단계

hello_dag — Airflow 3 의 import 두 줄

1. **from airflow.sdk import DAG**
   Airflow 3 에서 DAG 를 가져오는 정식 경로
2. **from airflow.providers.standard.operators.python import PythonOperator**
   옛 경로(airflow.operators.python)도 돌지만 DeprecatedImportWarning(실측)
3. **schedule="@daily" · catchup=False**
   start_date=datetime(2026, 1, 1) · tags=["week8"]
4. **af dags list | grep hello**
   hello_dag 줄이 보이고 is_paused 가 True 면 성공

> **파일은 cat > dags/hello_dag.py << 'EOF' … EOF 로 통째로 만든다**

1단계 · 언제 읽히나

새 파일은 최대 5분 — reserialize 면 바로

경우	읽히는 때
새 DAG 파일	dag-processor 가 5분마다 훑을 때 (refresh_interval 300초 — 실측)
af dags reserialize	지금 한 번 훑는다 — 약 20초(실측)
이미 있는 파일을 고침	30초 안에 다시 읽힌다

목록에 안 보이면 기다리기 전에 af dags reserialize 부터 — 새 파일을 지금 읽게 한다

2·3단계 — 일부러 실패시키기

구문 오류 파일을 넣어 보고, 태스크 하나만 돌려 본다

1. **echo "from airflow import DAG((((" > dags/broken.py**
   af dags reserialize 로 바로 읽힌다

2. **af dags list-import-errors**
   dags-folder · broken.py · SyntaxError: invalid syntax(실측) — 확인했으면 rm

3. **af tasks test hello_dag say_hello 2026-01-01**
   hello, airflow! 와 Task instance in success state — 약 14초(실측)

DAG 가 안 보일 때

가장 먼저 볼 곳

> **tasks test 는 DAG 를 켜지 않아도 돌고 스케줄에도 올리지 않는다 — 개발 중 가장 많이 쓰는 명령**

4·5단계

etl_practice — t1 >> t2 >> t3 세 태스크

extract	transform	load
1) 데이터 수집	2) 데이터 변환	3) 적재

schedule=None(수동 트리거 전용) · retries 1 · retry_delay 30초 — unpause 를 빼면 queued 에 머문다

af dags show 끝에 extract → transform · transform → load · unpause → trigger 뒤 list-runs 가 success(약 5초, 실측)

6·7단계 — 일부러 실패시키기

transform 에 raise 를 넣고 — 실패 지점부터만 다시

파일 고치기는 python3 - << 'EOF' 로 한 줄 바꿔 쓰기 — 어느 운영체제든 같습니다

raise 넣기	trigger	raise 빼기	tasks clear	다시 상태
grep -n raise sleep 35	sleep 60 뒤 상태 보기	grep -c raise 0 이면 됨	-t transform -d -y	세 줄 모두 success

6·7단계 · 실측

states-for-dag-run — 실패 때와 clear 뒤

태스크	6단계 — 실패	7단계 — clear 뒤
extract	success	success (다시 안 돎)
transform	failed — 30초 뒤 한 번 더 시도	success
load	upstream_failed — 시작 안 함	success

60초 전에 보면 transform 이 아직 up_for_retry(노랑)일 수 있다

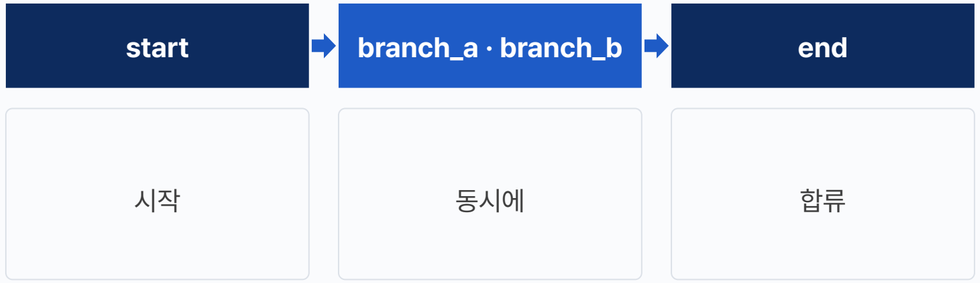
extract 의 start_date 는 6단계 시각 그대로, transform · load 만 방금 시각 — 실패한 자리부터 이어서 돈다

8·9단계

fan_practice — 갈라졌다 합류, 그리고 순환

start >> [a, b] >> end

네 줄 모두 success — A · B 시작이 0.001초도 안 되는 차이 (실측)



VS

end >> start 한 줄을 덧붙이면

sleep 40 뒤 af dags list-import-errors -o plain

- AirflowDagCycleException(실측)
- Faulty task: end — 순환은 강제로 거부
- 확인했으면 그 줄을 지워 되돌린다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. af dags list 에 hello_dag 가 나타났다
2. broken.py 가 list-import-errors 표에 잡혔고, 지웠다
3. tasks test 로그에 hello, airflow! 와 success 가 찍혔다
4. transform 실패 때 load 가 upstream_failed 로 멈췄다
5. clear 뒤 extract 는 다시 돌지 않고 세 줄 모두 success 가 됐다
6. 순환을 되돌려 마지막 줄이 start >> [a, b] >> end 다

8-2 · 실습 1.5H 마무리

다음은 유즈케이스 연구 (1H)

실무 사례 다섯을 훑고, 7주차에 손으로 하던 일을 태스크로 쪼개 8-3 의 골격 DAG 를 그립니다

- ✓ **배치 ETL·ML 재학습·리포트·품질 점검·RAG 갱신**
- ✓ **태스크마다 「멱등한가」 적기**
- ✓ **pipeline_skeleton 골격 DAG**
- ✓ **태스크 분해안 문서**

가져갈 것

dags/ 의 세 파일과 상태 표